# m2xhgr, SEAM M2XHGR

Giuseppe Silvi

April 4, 2026

| | |
|---|---|
| **author** | Giuseppe Silvi |
| **compile_options** | -lang cpp -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -ftz 0 |
| **description** | Mono to first-order spatial field via Haar wavelet decomposition |
| **filename** | m2xhgr.dsp |
| **license** | GPL-3.0 |
| **maths.lib/author** | GRAME |
| **maths.lib/copyright** | GRAME |
| **maths.lib/license** | LGPL with exception |
| **maths.lib/name** | Faust Math Library |
| **maths.lib/version** | 2.9.0 |
| **name** | m2xhgr, SEAM M2XHGR |
| **routes.lib/name** | Faust Signal Routing Library |
| **routes.lib/version** | 1.2.0 |
| **vendor** | SEAM |
| **version** | 0.2.0 |

This document provides a mathematical description of the Faust program text stored in the `m2xhgr.dsp` file. See the notice in Section 3 (page 4) for details.

## 1 Mathematical definition of `process`

The *m2xhgr* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals $y_i$ for $i \in [1, 4]$ such that

$$
\begin{aligned}
y_1(t) &= x(t) \\
y_2(t) &= p_{14}(t) \cdot s_7(t) - p_{11}(t) \cdot s_6(t) \\
y_3(t) &= p_8(t) \cdot s_4(t) + p_{15}(t) \cdot s_5(t) \\
y_4(t) &= p_{11}(t) \cdot s_7(t) + p_{14}(t) \cdot s_6(t)
\end{aligned}
$$

2. Input signal $x$
3. User-interface input signals $u_{si}$ for $i \in [1, 3]$ such that

| | | |
|---|---|---|
| "ROLL Rotation around X-axis" | $u_{s1}(t) \in [-180, 180]$ | (default value = 0) |
| "PITCH Rotation around Y-axis" | $u_{s2}(t) \in [-180, 180]$ | (default value = 0) |
| "YAW Rotation around Z-axis" | $u_{s3}(t) \in [-180, 180]$ | (default value = 0) |

4. Intermediate signals $p_i$ for $i \in [1, 15]$ and $s_i$ for $i \in [1, 7]$ such that

$$
\begin{aligned}
p_1(t) &= 0.0174532925199433 \cdot u_{s1}(t) \\
p_2(t) &= \cos(p_1(t)) \\
p_3(t) &= 0.25 \cdot p_2(t) \\
p_4(t) &= \sin(p_1(t)) \\
p_5(t) &= 0.5 \cdot p_4(t) \\
p_6(t) &= 0.0174532925199433 \cdot u_{s2}(t) \\
p_7(t) &= \sin(p_6(t)) \\
p_8(t) &= \cos(p_6(t)) \\
p_9(t) &= 0.25 \cdot p_8(t) \\
p_{10}(t) &= 0.0174532925199433 \cdot u_{s3}(t) \\
p_{11}(t) &= \sin(p_{10}(t)) \\
p_{12}(t) &= 0.25 \cdot p_4(t) \\
p_{13}(t) &= 0.5 \cdot p_2(t) \\
p_{14}(t) &= \cos(p_{10}(t)) \\
p_{15}(t) &= 0.25 \cdot p_7(t)
\end{aligned}
$$

$$
\begin{aligned}
s_1(t) &= x(t) + x(t-1) \\
s_2(t) &= (s_1(t) - s_1(t-1)) \\
s_3(t) &= (x(t) - x(t-1)) \\
s_4(t) &= (p_5(t) \cdot s_3(t) + p_3(t) \cdot s_2(t)) \\
s_5(t) &= (s_1(t) + s_1(t-1)) \\
s_6(t) &= (p_9(t) \cdot s_5(t) - p_7(t) \cdot s_4(t)) \\
s_7(t) &= (p_{13}(t) \cdot s_3(t) - p_{12}(t) \cdot s_2(t))
\end{aligned}
$$

## 2 Block diagram of `process`

The block diagram of `process` is shown on Figure 1 (page 4).

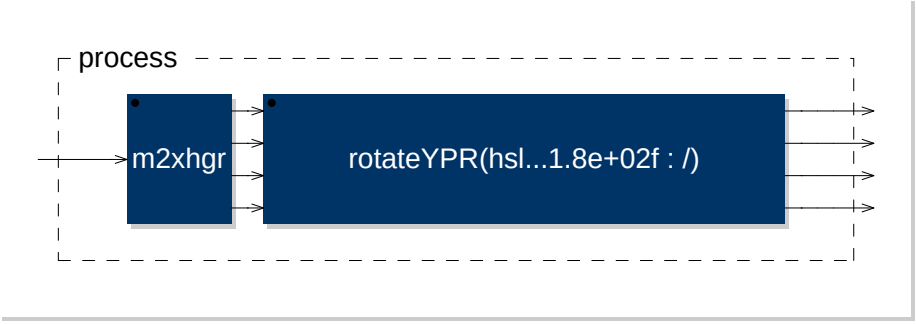


Figure 1: Block diagram of `process`

# 3 Notice

- This document was generated using Faust version 2.81.10 on April 04, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the $f_S$ sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal $s$ in $S$ is a function mapping[1] times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from $S^n$ to $S^m$, where $n, m \in \mathbb{N}$. See the Faust manual for additional information (http://faust.grame.fr).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E; each graph is put in a box labeled by I. Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `m2xhgr-mdoc/` directory may also include the following subdirectories:
  - `cpp/` for Faust compiled code;
  - `pdf/` which contains this document;
  - `src/` for all Faust sources used (even libraries);

[1] Faust assumes that $\forall\, s \in S, \forall\, t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

- `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (http://www.w3.org/Graphics/SVG/);
- `tex/` for the LaTeX source of this document.

# 4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `m2xhgr.dsp`

```
declare filename "m2xhgr.dsp";
declare name "m2xhgr";
declare name   "SEAM M2XHGR";
declare vendor "SEAM";
declare version "0.2.0";
declare author "Giuseppe Silvi";
declare license "GPL-3.0";
declare description "Mono to first-order spatial field via Haar wavelet decomposition";
//
// M2XHGR -Mono to first-order AmbiX via Haar wavelet
//
// Creates a 4-channel spatial representation from a mono signal
// using the Haar wavelet transform as a filter bank.
//
// The Haar wavelet is the simplest wavelet: a 2-point DFT applied
// sample-by-sample as a running filter pair (QMF):
//
//   sum(x) = (x[n] + x[n-1]) / 2 -lowpass (DC component)
//   dif(x) = (x[n] - x[n-1]) / 2 -highpass (AC component)
//
// Applied recursively, this decomposes the signal into frequency-ordered
// components. The reversed form (dft2r) outputs highpass first, so that
// the recursive QMF bank produces outputs ordered from highpass to
// lowpass -matching the ACN channel convention where negative-m
// harmonics (left side of the pyramid) carry high-frequency content
// and positive-m harmonics (right side) carry low-frequency content.
//
// haarmn(N) generates (N+1)^2 spatial components from a mono input:
//   - channel 0: passthrough (W, omnidirectional)
//   - channels 1 to (N+1)^2-1: Haar components, highpass to lowpass
//
// For N=1 (first order), the 4 outputs are:
//   a0 = passthrough (W -full spectrum, unfiltered)
//   a1 = dif        (Y - highpass, first-level difference)
//   a2 = dif_of_sum (Z - bandpass, second-level difference)
//   a3 = sum_of_sum (X - lowpass, second-level sum)
//
//
//
import("stdfaust.lib");
//
//             2-point DFT (Haar wavelet)
//
// Running sum/difference filter pair: the simplest QMF.
// Standard form: lowpass first, highpass second.
//
dft2(x) = s(x), d(x)
with {
   s(x) = (x + x') / 2; // lowpass (DC)
```

```
    d(x) = (x - x') / 2; // highpass (AC)
};
//
// Reversed form: highpass first, lowpass second.
// Used in the QMF bank so that outputs are ordered high-to-low,
// matching the ACN channel convention.
//
dft2r(x) = dft2(x) : ro.cross(2);
//
//             Recursive QMF filter bank
//
// qmf(N) applies dft2r recursively N times, producing N+1 outputs.
// At each level, the highpass component exits and the lowpass
// continues into the next decomposition. The result is a set of
// sub-band signals ordered from highest to lowest frequency.
//
qmf(0) = _;
qmf(1) = dft2r;
qmf(N) = dft2r : _, qmf(N-1);
//
//             Haar spatial decomposition
//
// haarmn(N): mono input to (N+1)^2 spatial components.
// The passthrough (unfiltered signal) occupies channel 0 (W),
// followed by the QMF sub-bands as directional components.
//
haarmn(0) = _;
haarmn(N) = _ <: _, qmf((N+1)^2-2);
//
// m2xhgr: mono to 4-channel first-order spatial field
//
m2xhgr = haarmn(1);
//
//              First-order AmbiX rotation
//
// Applies yaw, pitch, roll rotation to a first-order signal.
// Channel 0 (W) passes through unchanged.
//
rotateYPR(yaw, pitch, roll, a0, a1, a2, a3) =
    a0, a1_ypr, a2_ypr, a3_ypr
with {
    a1_r = a1*cos(roll) - a2*sin(roll);
    a2_r = a1*sin(roll) + a2*cos(roll);
    a3_r = a3;
    a1_rp = a1_r;
    a2_rp = a2_r*cos(pitch) + a3_r*sin(pitch);
    a3_rp = -a2_r*sin(pitch) + a3_r*cos(pitch);
    a1_ypr = a1_rp*cos(yaw) - a3_rp*sin(yaw);
    a2_ypr = a2_rp;
    a3_ypr = a1_rp*sin(yaw) + a3_rp*cos(yaw);
};
//
//              Parameters
//
yaw  = hslider("[00]YAW Rotation around Z-axis",0,-180,180,0.5) : ma.deg2rad;
pitch = hslider("[01]PITCH Rotation around Y-axis",0,-180,180,0.5) : ma.deg2rad;
roll = hslider("[02]ROLL Rotation around X-axis",0,-180,180,0.5) : ma.deg2rad;
//
//              Process
//
// Mono input →Haar decomposition →rotation →4-channel AmbiX output
//
process = m2xhgr : rotateYPR(yaw, pitch, roll);
```

Listing 2: `stdfaust.lib`

```
//################################ stdfaust.lib ##########################################
// The purpose of this library is to give access to all the Faust standard libraries
// through a series of environments.
//#######################################################################################

aa = library("aanl.lib");
sf = library("all.lib");
an = library("analyzers.lib");
ba = library("basics.lib");
co = library("compressors.lib");
de = library("delays.lib");
dm = library("demos.lib");
dx = library("dx7.lib");
en = library("envelopes.lib");
fd = library("fds.lib");
fi = library("filters.lib");
ho = library("hoa.lib");
it = library("interpolators.lib");
la = library("linearalgebra.lib");
ma = library("maths.lib");
mi = library("mi.lib");
ef = library("misceffects.lib");
os = library("oscillators.lib");
no = library("noises.lib");
pf = library("phaflangers.lib");
pl = library("platform.lib");
pm = library("physmodels.lib");
qu = library("quantizers.lib");
rm = library("reducemaps.lib");
re = library("reverbs.lib");
ro = library("routes.lib");
sp = library("spats.lib");
si = library("signals.lib");
so = library("soundfiles.lib");
sy = library("synths.lib");
ve = library("vaeffects.lib");
vl = library("version.lib");
wa = library("webaudio.lib");
wd = library("wdmodels.lib");
```

Listing 3: `maths.lib`

```
//################################### maths.lib ##########################################
// Mathematic library for Faust. Its official prefix is `ma`.
//
// #### References
// * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
//#######################################################################################
// Some functions are implemented as Faust foreign functions of `math.h` functions
// that are not part of Faust's primitives. Defines also various constants and several
// utilities.
//#######################################################################################

// ## History
// * 06/13/2016 [RM] normalizing and integrating to new libraries
// * 07/08/2015 [YO] documentation comments
// * 20/06/2014 [SL] added FTZ function
// * 22/06/2013 [YO] added float|double|quad variants of some foreign functions
// * 28/06/2005 [YO] postfixed functions with 'f' to force float version instead of double
// * 28/06/2005 [YO] removed 'modf' because it requires a pointer as argument

/*************************************************************************
```

```
************************************************************************
FAUST library file
Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
------------------------------------------------------------------------
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU Lesser General Public License as
published by the Free Software Foundation; either version 2.1 of the
License, or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with the GNU C Library; if not, write to the Free
Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
02111-1307 USA.

EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
larger FAUST program which directly or indirectly imports this library
file and still distribute the compiled code generated by the FAUST
compiler, or a modified version of this compiled code, under your own
copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
grants you the right to freely choose the license for the resulting
compiled code. In particular the resulting compiled code has no obligation
to be LGPL or GPL. For example you are free to choose a commercial or
closed source license or any other license if you decide so.
************************************************************************
************************************************************************/

// This library contains platform specific constants
ba = library("basics.lib");
pl = library("platform.lib");
ma = library("maths.lib"); // for compatible copy/paste out of this file

declare name "Faust Math Library";
declare version "2.9.0";
declare author "GRAME";
declare copyright "GRAME";
declare license "LGPL with exception";

//============================Functions Reference=========================================
//========================================================================================


//---------------------------------`(ma.)SR`--------------------------------------
// Current sampling rate given at init time. Constant during program execution.
//
// #### Usage
//
// ```
// SR : _
// ```
//--------------------------------------------------------------------------------
SR = pl.SR;

//---------------------------------`(ma.)T`----------------------------------------
// Current sample duration in seconds computed from the sampling rate given at init time.
      Constant during program execution.
//
// #### Usage
//
// ```
// T : _
// ```
//--------------------------------------------------------------------------------
T = 1.0 / SR;
```

```

//---------------------------------`(ma.)BS`--------------------------------------
// Current block-size. Can change during the execution at each block.
//
// #### Usage
//
// ```
// BS : _
// ```
//--------------------------------------------------------------------------
BS = pl.BS;


//---------------------------------`(ma.)PI`--------------------------------------
// Constant PI in double precision.
//
// #### Usage
//
// ```
// PI : _
// ```
//--------------------------------------------------------------------------
PI = 3.14159265358979323846;


//---------------------------------`(ma.)deg2rad`----------------------------------
// Convert degrees to radians.
//
// #### Usage
//
// ```
// 45. : deg2rad
// ```
//--------------------------------------------------------------------------
deg2rad = _ * PI / 180.;


//---------------------------------`(ma.)rad2deg`----------------------------------
// Convert radians to degrees.
//
// #### Usage
//
// ```
// ma.PI : rad2deg
// ```
//--------------------------------------------------------------------------
rad2deg = _ * 180. / PI;


//---------------------------------`(ma.)E`-----------------------------------------
// Constant e in double precision.
//
// #### Usage
//
// ```
// E : _
// ```
//--------------------------------------------------------------------------
E = 2.71828182845904523536;


//---------------------------------`(ma.)EPSILON`-----------------------------------------
// Constant EPSILON available in simple/double/quad precision,
// as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
// and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
// that is smallest positive number such that `1.0 + EPSILON != 1.0`.
//
// #### Usage
```

```
//
// ```
// EPSILON : _
// ```
//-----------------------------------------------------------------------------
singleprecision EPSILON = 1.192092896e-07;
doubleprecision EPSILON = 2.2204460492503131e-016;
quadprecision EPSILON = 1.084202172485504434007452e-019;
fixedpointprecision EPSILON = 2.2204460492503131e-016;


//---------------------------------`(ma.)MIN`--------------------------------------
// Constant MIN available in simple/double/quad precision (minimal positive value).
//
// #### Usage
//
// ```
// MIN : _
// ```
//-----------------------------------------------------------------------------
singleprecision MIN = 1.175494351e-38;
doubleprecision MIN = 2.2250738585072014e-308;
quadprecision MIN = 2.2250738585072014e-308;
fixedpointprecision MIN = 2.2250738585072014e-308;


//---------------------------------`(ma.)MAX`-----------------------------
// Constant MAX available in simple/double/quad precision (maximal positive value).
//
// #### Usage
//
// ```
// MAX : _
// ```
//-----------------------------------------------------------------------------
singleprecision MAX = 3.402823466e+38;
doubleprecision MAX = 1.7976931348623158e+308;
quadprecision MAX = 1.7976931348623158e+308;
fixedpointprecision MAX = 1.7976931348623158e+308;

// Obsolete, kept for compatibility reasons
INFINITY = MAX;

//---------------------------------`(ma.)FTZ`--------------------------------------
// Flush to zero: force samples under the "maximum subnormal number"
// to be zero. Usually not needed in C++ because the architecture
// file take care of this, but can be useful in JavaScript for instance.
//
// #### Usage
//
// ```
// _ : FTZ : _
// ```
//
// #### Reference
//
// <http://docs.oracle.com/cd/E19957-01/806-3568/ncg_math.html>
//-----------------------------------------------------------------------------
FTZ(x) = x * (abs(x) > MIN);


//---------------------------------`(ma.)copysign`----------------------------------------
// Changes the sign of x (first input) to that of y (second input).
//
// #### Usage
//
// ```
// _,_ : copysign : _
```

```
// ```
//-----------------------------------------------------------------------------
copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");


//---------------------------------`(ma.)neg`--------------------------------------
// Invert the sign (-x) of a signal.
//
// #### Usage
//
// ```
// _ : neg : _
// ```
//-----------------------------------------------------------------------------
neg(x) = -x;


//---------------------------------`(ma.)not`--------------------------------------
// Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-primitive) as `not(x) = x xor -1;`.
// So working regardless of the size of the integer, assuming negative numbers in two's complement.
//
// #### Usage
//
// ```
// _ : not : _
// ```
//-----------------------------------------------------------------------------
not(x) = x xor -1;


//-------`(ma.)sub(x,y)`------------------
// Subtract `x` and `y`.
//
// #### Usage
//
// ```
// _,_ : sub : _
// ```
//------------------------------
sub(x,y) = y-x;


//---------------------------------`(ma.)inv`--------------------------------------
// Compute the inverse (1/x) of the input signal.
//
// #### Usage
//
// ```
// _ : inv : _
// ```
//-----------------------------------------------------------------------------
inv(x) = 1/x;


//---------------------------------`(ma.)cbrt`--------------------------------------
// Computes the cube root of of the input signal.
//
// #### Usage
//
// ```
// _ : cbrt : _
// ```
//-----------------------------------------------------------------------------
cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");


```

```
//----------------------------------`(ma.)hypot`-------------------------------------
// Computes the euclidian distance of the two input signals
// sqrt(x*x+y*y) without undue overflow or underflow.
//
// #### Usage
//
// ```
// _,_ : hypot : _
// ```
//--------------------------------------------------------------------------------
hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");


//----------------------------------`(ma.)ldexp`-------------------------------------
// Takes two input signals: x and n, and multiplies x by 2 to the power n.
//
// #### Usage
//
// ```
// _,_ : ldexp : _
// ```
//--------------------------------------------------------------------------------
ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");


//----------------------------------`(ma.)scalb`-------------------------------------
// Takes two input signals: x and n, and multiplies x by 2 to the power n.
//
// #### Usage
//
// ```
// _,_ : scalb : _
// ```
//--------------------------------------------------------------------------------
scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");


//----------------------------------`(ma.)log1p`-------------------------------------
// Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
//
// #### Usage
//
// ```
// _ : log1p : _
// ```
//--------------------------------------------------------------------------------
log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");


//----------------------------------`(ma.)logb`-------------------------------------
// Return exponent of the input signal as a floating-point number.
//
// #### Usage
//
// ```
// _ : logb : _
// ```
//--------------------------------------------------------------------------------
logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");


//----------------------------------`(ma.)ilogb`-------------------------------------
// Return exponent of the input signal as an integer number.
//
// #### Usage
//
// ```
// _ : ilogb : _
```

```
// ```
//-----------------------------------------------------------------------------
ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");


//---------------------------------`(ma.)log2`-------------------------------------
// Returns the base 2 logarithm of x.
//
// #### Usage
//
// ```
// _ : log2 : _
// ```
//-----------------------------------------------------------------------------
log2(x) = log(x)/log(2.0);


//---------------------------------`(ma.)expm1`-------------------------------------
// Return exponent of the input signal minus 1 with better precision.
//
// #### Usage
//
// ```
// _ : expm1 : _
// ```
//-----------------------------------------------------------------------------
expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");


//---------------------------------`(ma.)acosh`-------------------------------------
// Computes the principle value of the inverse hyperbolic cosine
// of the input signal.
//
// #### Usage
//
// ```
// _ : acosh : _
// ```
//-----------------------------------------------------------------------------
acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");


//---------------------------------`(ma.)asinh`-------------------------------------
// Computes the inverse hyperbolic sine of the input signal.
//
// #### Usage
//
// ```
// _ : asinh : _
// ```
//-----------------------------------------------------------------------------
asinh = ffunction(float asinhf|asinh|asinhl (float), <math.h>, "");


//---------------------------------`(ma.)atanh`-------------------------------------
// Computes the inverse hyperbolic tangent of the input signal.
//
// #### Usage
//
// ```
// _ : atanh : _
// ```
//-----------------------------------------------------------------------------
atanh = ffunction(float atanhf|atanh|atanhl (float), <math.h>, "");


//---------------------------------`(ma.)sinh`----------------------------------------
// Computes the hyperbolic sine of the input signal.
```

```
//
// #### Usage
//
// ```
// _ : sinh : _
// ```
//-----------------------------------------------------------------------------
sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");


//---------------------------------`(ma.)cosh`-------------------------------------
// Computes the hyperbolic cosine of the input signal.
//
// #### Usage
//
// ```
// _ : cosh : _
// ```
//-----------------------------------------------------------------------------
cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");


//---------------------------------`(ma.)tanh`-------------------------------------
// Computes the hyperbolic tangent of the input signal.
//
// #### Usage
//
// ```
// _ : tanh : _
// ```
//-----------------------------------------------------------------------------
tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");


//---------------------------------`(ma.)erf`--------------------------------------
// Computes the error function of the input signal.
//
// #### Usage
//
// ```
// _ : erf : _
// ```
//-----------------------------------------------------------------------------
erf = ffunction(float erff|erf|erfl(float), <math.h>,"");


//---------------------------------`(ma.)erfc`-------------------------------------
// Computes the complementary error function of the input signal.
//
// #### Usage
//
// ```
// _ : erfc : _
// ```
//-----------------------------------------------------------------------------
erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");


//---------------------------------`(ma.)gamma`------------------------------------
// Computes the gamma function of the input signal.
//
// #### Usage
//
// ```
// _ : gamma : _
// ```
//-----------------------------------------------------------------------------
gamma = ffunction(float tgammaf|tgamma|tgammal(float), <math.h>,"");
```

```
//--------------------------------`(ma.)lgamma`-----------------------------------
// Calculates the natural logorithm of the absolute value of
// the gamma function of the input signal.
//
// #### Usage
//
// ```
// _ : lgamma : _
// ```
//--------------------------------------------------------------------------
lgamma = ffunction(float lgammaf|lgamma|lgammal(float), <math.h>,"");


//---------------------------------`(ma.)J0`--------------------------------------
// Computes the Bessel function of the first kind of order 0
// of the input signal.
//
// #### Usage
//
// ```
// _ : J0 : _
// ```
//--------------------------------------------------------------------------
J0 = ffunction(float j0(float), <math.h>,"");


//---------------------------------`(ma.)J1`--------------------------------------
// Computes the Bessel function of the first kind of order 1
// of the input signal.
//
// #### Usage
//
// ```
// _ : J1 : _
// ```
//--------------------------------------------------------------------------
J1 = ffunction(float j1(float), <math.h>,"");


//---------------------------------`(ma.)Jn`--------------------------------------
// Computes the Bessel function of the first kind of order n
// (first input signal) of the second input signal.
//
// #### Usage
//
// ```
// _,_ : Jn : _
// ```
//--------------------------------------------------------------------------
Jn = ffunction(float jn(int, float), <math.h>,"");


//---------------------------------`(ma.)Y0`--------------------------------------
// Computes the linearly independent Bessel function of the second kind
// of order 0 of the input signal.
//
// #### Usage
//
// ```
// _ : Y0 : _
// ```
//--------------------------------------------------------------------------
Y0 = ffunction(float y0(float), <math.h>,"");


//---------------------------------`(ma.)Y1`--------------------------------------
```

```
// Computes the linearly independent Bessel function of the second kind
// of order 1 of the input signal.
//
// #### Usage
//
// ```
// _ : Y0 : _
// ```
//-----------------------------------------------------------------------------
Y1 = ffunction(float y1(float), <math.h>,"");


//----------------------------------`(ma.)Yn`---------------------------------------
// Computes the linearly independent Bessel function of the second kind
// of order n (first input signal) of the second input signal.
//
// #### Usage
//
// ```
// _,_ : Yn : _
// ```
//-----------------------------------------------------------------------------
Yn = ffunction(float yn(int, float), <math.h>,"");


//----------------------------`(ma.)fabs`, `(ma.)fmax`, `(ma.)fmin
      `---------------------------
// Just for compatibility...
//
// ```
// fabs = abs
// fmax = max
// fmin = min
// ```
//-----------------------------------------------------------------------------
fabs = abs;
fmax = max;
fmin = min;

//-------------------------------`(ma.)np2`--------------------------------------
// Gives the next power of 2 of x.
//
// #### Usage
//
// ```
// np2(n) : _
// ```
//
// Where:
//
// * `n`: an integer
//-----------------------------------------------------------------------------
np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);


//----------------------------`(ma.)frac`--------------------------------------
// Gives the fractional part of n.
//
// #### Usage
//
// ```
// frac(n) : _
// ```
//
// Where:
//
// * `n`: a decimal number
//-----------------------------------------------------------------------------
```

```
frac(n) = n - floor(n);
decimal = frac;

// NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
// in filters.lib so we decided to keep that one... decimal is declared though for
// backward compatibility.
// decimal(n) = n - floor(n);


//-------------------------------`(ma.)modulo`--------------------------------------
// Modulus operation using the `(x%y+y)%y` formula to ensures the result is always non-
    negative, even if `x` is negative.
//
// #### Usage
//
// ```
// modulo(x,y) : _
// ```
//
// Where:
//
// * `x`: the numerator
// * `y`: the denominator
//--------------------------------------------------------------------------------
modulo(x,y) = (x % y + y) % y;


//---------------`(ma.)isnan`----------------
// Return non-zero if x is a NaN.
//
// #### Usage
//
// ```
// isnan(x)
// _ : isnan : _
// ```
//
// Where:
//
// * `x`: signal to analyse
//-----------------------------------------
isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");


//---------------`(ma.)isinf`----------------
// Return non-zero if x is a positive or negative infinity.
//
// #### Usage
//
// ```
// isinf(x)
// _ : isinf : _
// ```
//
// Where:
//
// * `x`: signal to analyse
//-----------------------------------------
isinf = ffunction(int isinff|isinf|isinfl (float),<math.h>,"");

nextafter = ffunction(float nextafter(float, float),<math.h>,"");


//---------------------------`(ma.)chebychev`------------------------------
// Chebychev transformation of order N.
//
// #### Usage
//
```

```
// ```
// _ : chebychev(N) : _
// ```
//
// Where:
//
// * `N`: the order of the polynomial, a constant numerical expression
//
// #### Semantics
//
// ```
// T[0](x) = 1,
// T[1](x) = x,
// T[n](x) = 2x*T[n-1](x) - T[n-2](x)
// ```
//
// #### Reference
//
// <http://en.wikipedia.org/wiki/Chebyshev_polynomial>
//----------------------------------------------------------------------
chebychev(0,x) = 1;
chebychev(1,x) = x;
chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);


//------------------------`(ma.)chebychevpoly`-------------------------------
// Linear combination of the first Chebyshev polynomials.
//
// #### Usage
//
// ```
// _ : chebychevpoly((c0,c1,...,cn)) : _
// ```
//
// Where:
//
// * `cn`: the different Chebychevs polynomials such that:
// chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
//
// #### Reference
//
// <http://www.csounds.com/manual/html/chebyshevpoly.html>
//----------------------------------------------------------------------
chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
   with {
      L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
      L(n,c)     = chebychev(n)*c;
   };


//------------------`(ma.)diffn`----------------------------
// Negated first-order difference.
//
// #### Usage
//
// ```
// _ : diffn : _
// ```
//-------------------------------------------------------
diffn(x) = x' - x; // negated first-order difference


//------------------`(ma.)signum`----------------------------
// The signum function signum(x) is defined as
// -1 for x<0, 0 for x==0, and 1 for x>0.
//
// #### Usage
//
```

```
// ```
// _ : signum : _
// ```
//-------------------------------------------------------
signum(x) = (x>0)-(x<0);


//------------------`(ma.)nextpow2`----------------------------
// The nextpow2(x) returns the lowest integer m such that
// 2^m >= x.
//
// #### Usage
//
// ```
// 2^nextpow2(n) : _
// ```
// Useful for allocating delay lines, e.g.,
// ```
// delay(2^nextpow2(maxDelayNeeded), currentDelay);
// ```
//-------------------------------------------------------
nextpow2(x) = ceil(log(x)/log(2.0));


//--------------------`(ma.)zc`-----------------------------------------------
// Indicator function for zero-crossing: it returns 1 if a zero-crossing
// occurs, 0 otherwise.
//
// #### Usage
//
// ```
// _ : zc : _
// ```
//-----------------------------------------------------------------------------
zc = ba.tAndH(!=(0)) <: *(mem) < 0;

//-------------------------`(ma.)unwrap` --------------------------------------
// Unwrap the input signal so that successive output values never differ
// by more than `pi`, switching to a 2*pi-complementary value when needed.
//
// #### Usage
//
// ```
// _ : unwrap(pi) : _
// ```
//
// Where:
//
// * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
//
// #### Example test program
//
// ```
// process = 0 - os.oscrc(1000)    // the true phase is either -PI or +PI
//       : an.resonator(1,1000) : !,_ // oscillates between -PI and +PI
//       : ma.unwrap(ma.PI);      // oscillates near +PI
// ```
//
//-----------------------------------------------------------------------------
unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
    dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi),-(2*pi));
    wrap(x,d) = x - d * floor(x / d);
};

//--------------------`(ma.)primes`-----------------------------------------------
// Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
// primes(1) is 3, and so on. The waveform is length 2048, so the largest
// precomputed prime is primes(2047) which is 17863.
```

```
//
// #### Usage
//
// ```
// _ : primes : _
// ```
//------------------------------------------------------------------------------
primes(x) = rdtable(waveform{
2,3,5,7,11,13,17,19,23,29,
31,37,41,43,47,53,59,61,67,71,
73,79,83,89,97,101,103,107,109,113,
127,131,137,139,149,151,157,163,167,173,
179,181,191,193,197,199,211,223,227,229,
233,239,241,251,257,263,269,271,277,281,
283,293,307,311,313,317,331,337,347,349,
353,359,367,373,379,383,389,397,401,409,
419,421,431,433,439,443,449,457,461,463,
467,479,487,491,499,503,509,521,523,541,
547,557,563,569,571,577,587,593,599,601,
607,613,617,619,631,641,643,647,653,659,
661,673,677,683,691,701,709,719,727,733,
739,743,751,757,761,769,773,787,797,809,
811,821,823,827,829,839,853,857,859,863,
877,881,883,887,907,911,919,929,937,941,
947,953,967,971,977,983,991,997,1009,1013,
1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
```

```
4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
```

```
10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
```

```
17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
17789,17791,17807,17827,17837,17839,17851,17863
}, x);
```

Listing 4: `routes.lib`

```
//#################################### routes.lib #######################################
// A library to handle signal routing in Faust. Its official prefix is `ro`.
//
// #### References
// * <https://github.com/grame-cncm/faustlibraries/blob/master/routes.lib>
//#######################################################################################

/************************************************************************
*************************************************************************
FAUST library file
Copyright (C) 2003-2019 GRAME, Centre National de Creation Musicale
------------------------------------------------------------------------
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU Lesser General Public License as
published by the Free Software Foundation; either version 2.1 of the
License, or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with the GNU C Library; if not, write to the Free
Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
02111-1307 USA.

EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
larger FAUST program which directly or indirectly imports this library
file and still distribute the compiled code generated by the FAUST
compiler, or a modified version of this compiled code, under your own
copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
grants you the right to freely choose the license for the resulting
compiled code. In particular the resulting compiled code has no obligation
to be LGPL or GPL. For example you are free to choose a commercial or
closed source license or any other license if you decide so.
************************************************************************
************************************************************************/

ba = library("basics.lib");
si = library("signals.lib");
sp = library("spats.lib");

declare name "Faust Signal Routing Library";
declare version "1.2.0";

//============================Functions Reference=========================================
//=======================================================================================

//--------------------------------`(ro.)cross`-----------------------------------
// Cross N signals: `(x1,x2,..,xn) -> (xn,..,x2,x1)`.
```

```
// `cross` is a standard Faust function.
//
// #### Usage
//
// ```
// cross(N)
// _,_,_ : cross(3) : _,_,_
// ```
//
// Where:
//
// * `N`: number of signals (int, as a constant numerical expression)
//
// #### Note
//
// Special case: `cross2`:
//
// ```
// cross2 = _,cross(2),_;
// ```
//------------------------------------------------------------------------------
// cross n cables : (x1,x2,..,xn) -> (xn,..,x2,x1)
cross(N) = route(N, N, par(i, N, (i+1, N-i)));
cross2 = _,cross(2),_; // for compatibility with some old misceffects.lib functions


//--------------`(ro.)crossnn`--------------
// Cross two `bus(N)`s.
//
// #### Usage
//
// ```
// (si.bus(2*N)) : crossnn(N) : (si.bus(2*N))
// ```
//
// Where:
//
// * `N`: the number of signals in the `bus` (int, as a constant numerical expression)
//--------------------------------------
crossnn(N) = crossNM(N,N);


//--------------`(ro.)crossn1`--------------
// Cross `bus(N)` and `bus(1)`.
//
// #### Usage
//
// ```
// (si.bus(N),_) : crossn1(N) : (_,si.bus(N))
// ```
//
// Where:
//
// * `N`: the number of signals in the first `bus` (int, as a constant numerical expression)
//--------------------------------------
crossn1(N) = crossNM(N,1);


//--------------`(ro.)cross1n`--------------
// Cross `bus(1)` and `bus(N)`.
//
// #### Usage
//
// ```
// (_,si.bus(N)) : crossn1(N) : (si.bus(N),_)
// ```
//
// Where:
```

```
//
// * `N`: the number of signals in the second `bus` (int, as a constant numerical expression)
//--------------------------------------
cross1n(N) = crossNM(1,N);


//--------------`(ro.)crossNM`--------------
// Cross `bus(N)` and `bus(M)`.
//
// #### Usage
//
// ```
// (si.bus(N),si.bus(M)) : crossNM(N,M) : (si.bus(M),si.bus(N))
// ```
//
// Where:
//
// * `N`: the number of signals in the first `bus` (int, as a constant numerical expression)
// * `M`: the number of signals in the second `bus` (int, as a constant numerical expression)
//--------------------------------------
crossNM(N,M) = route(N+M, N+M, par(i, N+M, i+1, ((i+M)%(N+M))+1));


//-------------------------`(ro.)interleave`-----------------------------
// Interleave R x C cables from column order to row order. That is, transpose the input CxR matrix,
// the first R inputs is the first row.
//
// input : `x(0), x(1), x(2) ..., x(row*col-1)`
//
// output: `x(0+0*row), x(0+1*row), x(0+2*row), ..., x(1+0*row), x(1+1*row), x(1+2*row), ...`
//
//
//
// #### Usage
//
// ```
// si.bus(R*C) : interleave(R,C) : si.bus(R*C)
// ```
//
// Where:
//
// * `R`: row length (int, as a constant numerical expression)
// * `C`: column length (int, as a constant numerical expression)
//--------------------------------------------------------------------------
interleave(1,2) = _,_;
interleave(R,C) = route(R*C, R*C, par(i, R*C, (i+1, (i%R)*C + int(i/R) + 1)));

//------------------------------`(ro.)butterfly`--------------------------------
// Addition (first half) then substraction (second half) of interleaved signals.
//
// #### Usage
//
// ```
// si.bus(N) : butterfly(N) : si.bus(N)
// ```
//
// Where:
//
// * `N`: size of the butterfly (N is int, even and as a constant numerical expression)
//--------------------------------------------------------------------------
butterfly(2) = si.bus(2) <: +,-;
butterfly(N) = si.bus(N) <: interleave(N/2,2), interleave(N/2,2) : par(i, N/2, +), par(i, N/2, -);


//------------------------------`(ro.)hadamard`----------------------------------
// Hadamard matrix function of size `N = 2^k`.
```

```
//
// #### Usage
//
// ```
// si.bus(N) : hadamard(N) : si.bus(N)
// ```
//
// Where:
//
// * `N`: `2^k`, size of the matrix (int, as a constant numerical expression)
//
//------------------------------------------------------------------------------
declare hadamard author "Remy Muller, revised by Romain Michon";
hadamard(2) = butterfly(2);
hadamard(N) = butterfly(N) : (hadamard(N/2), hadamard(N/2));


//---------------`(ro.)recursivize`-------------
// Create a recursion from two arbitrary processors `p` and `q`.
//
// #### Usage
//
// ```
// _,_ : recursivize(p,q) : _,_
//
// ```
//
// Where:
//
// * `p`: the forward arbitrary processor
// * `q`: the feedback arbitrary processor
//----------------------------------------
recursivize(p,q) = (_,_,_,_ :> sp.stereoize(p)) ~ sp.stereoize(q);


//--------------------`(ro.)bubbleSort`----------------------------------------
//
// Sort a set of N parallel signals in ascending order on-the-fly through
// the Bubble Sort algorithm.
//
// Mechanism: having a set of N parallel signals indexed from 0 to N - 1,
// compare the first pair of signals and swap them if sig[0] > sig[1];
// repeat the pair comparison for the signals sig[1] and sig[2], then again
// recursively until reaching the signals sig[N - 2] and sig[N - 1]; by the end,
// the largest element in the set will be placed last; repeat the process for
// the remaining N - 1 signals until there is a single pair left.
//
// Note that this implementation will always perform the worst-case
// computation, O(n^2).
//
// Even though the Bubble Sort algorithm is one of the least efficient ones,
// it is a useful example of how automatic sorting can be implemented at the
// signal level.
//
// #### Usage
//
// ```
// si.bus(N) : bubbleSort(N) : si.bus(N)
//
// ```
//
// Where:
//
// * `N`: the number of signals to be sorted (must be an int >= 0, as a constant numerical
      expression)
//
// #### Reference
// <https://en.wikipedia.org/wiki/Bubble_sort>
```

```
//-----------------------------------------------------------------------------
declare bubbleSort author "Dario Sanfilippo";
declare bubbleSort copyright "Copyright (C) 2021 Dario Sanfilippo
    <sanfilippo.dario@gmail.com>";
declare bubbleSort license "MIT License";
bubbleSort(0) = 0 : !;
bubbleSort(1) = _;
bubbleSort(N) = seq(i, N - 1, pairSortN(N - i), bus(i))
    with {
        bus(0) = 0 : !;
        bus(N) = si.bus(N);
        pairSort = bus(2) <: select2(>), select2(<);
        pairSortN(N) = seq(i, N - 1, bus(i), pairSort, bus(N - i - 2));
    };
```